

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ**

Факультет прикладной информатики

Образовательная программа Мобильные и сетевые технологии

Направление подготовки 09.03.03 Мобильные и сетевые технологии

О Т Ч Е Т

Лабораторная работа №5

Тема работы: «Процедуры, функции, триггеры в PostgreSQL»

Обучающийся: Федорова Мария Витальевна, К3239

Поток: ПиРБД К24 МСТ 1.2

Преподаватель: Говорова М.М.

Санкт-Петербург,
2026

1. Цель работы

Овладеть практическими навыками создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

2. Практическое задание

В ходе выполнения лабораторной работы требовалось:

1. создать 3 процедуры для индивидуальной базы данных согласно варианту;
2. создать триггеры для индивидуальной базы данных согласно варианту;
3. для варианта 2.1 реализовать 10 триггеров по индивидуальному заданию;
4. выполнить работу в консоли `psql`;
5. подтвердить создание и работу процедур, функций и триггеров скриншотами.

Индивидуальный вариант: **вариант 13, база данных «Ресторан»**. Для данного варианта были реализованы процедуры для вывода заказов официанта за дату, расчета стоимости заказа и повышения оклада сотрудника при изменении категории.

3. Сведения об индивидуальной базе данных

В работе использовалась база данных `restaurant_service`, разработанная ранее для варианта «Ресторан». Работа выполнялась в схеме `restaurant`. Основные таблицы, использованные при создании процедур и триггеров: `customer_order`, `order_item`, `dish`, `dish_ingredient`, `ingredient`, `purchase`, `restaurant_table`, `cooking_assignment`, `waiter`, `cook`, `head_chef`, `employee`.

4. Выполнение работы

4.1. Создание процедур

В соответствии с индивидуальным заданием были созданы три процедуры:

1. `pr_lr5_waiter_orders_by_date`;
2. `pr_lr5_calculate_order_cost`;
3. `pr_lr5_raise_employee_category_salary`.

Процедура 1. Вывод заказов официанта за заданную дату

Первая процедура предназначена для получения сведений о заказах конкретного официанта за заданную дату. В качестве параметров передаются идентификатор официанта и дата. Результат возвращается через курсор, поэтому вызов процедуры выполняется внутри транзакции.

```
restaurant_service=# CREATE OR REPLACE PROCEDURE pr_lr5_waiter_orders_by_date(
restaurant_service(# IN p_id_waiter INTEGER,
restaurant_service(# IN p_order_date DATE,
restaurant_service(# INOUT p_result REFCURSOR DEFAULT 'cur_waiter_orders'
restaurant_service(# )
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service$# BEGIN
restaurant_service$#     IF NOT EXISTS (
restaurant_service$#         SELECT 1
restaurant_service$#         FROM waiter
restaurant_service$#         WHERE id_waiter = p_id_waiter
restaurant_service$#     ) THEN
restaurant_service$#         RAISE EXCEPTION 'Официант с ID % не найден', p_id_waiter;
restaurant_service$#     END IF;
restaurant_service$#     OPEN p_result FOR
restaurant_service$#         SELECT
restaurant_service$#             co.id_order,
restaurant_service$#             co.order_datetime::date AS order_date,
restaurant_service$#             co.order_datetime::time AS order_time,
restaurant_service$#             co.id_restaurant_table,
restaurant_service$#             co.id_customer,
restaurant_service$#             co.status,
restaurant_service$#             co.wishes,
restaurant_service$#             co.markup,
restaurant_service$#             co.total_price
restaurant_service$#         FROM customer_order co
restaurant_service$#         WHERE co.id_waiter = p_id_waiter
restaurant_service$#             AND co.order_datetime::date = p_order_date
restaurant_service$#         ORDER BY co.order_datetime, co.id_order;
restaurant_service$# END;
restaurant_service=# $$;
CREATE PROCEDURE
restaurant_service=#
```

Рис. 1. Код процедуры pr_lr5_waiter_orders_by_date

На рисунке показано создание процедуры. Внутри процедуры проверяется существование официанта, после чего открывается курсор с заказами за указанную дату.

```
restaurant_service=# SELECT
restaurant_service=#     id_order,
restaurant_service=#     id_waiter,
restaurant_service=#     order_datetime::date AS order_date,
restaurant_service=#     status,
restaurant_service=#     total_price
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_waiter = 1
restaurant_service=# ORDER BY id_order;
 id_order | id_waiter | order_date | status | total_price
-----+-----+-----+-----+-----
      1 |      1 | 2026-03-20 | принят |      980.00
      2 |      1 | 2026-03-20 | готовится |      735.00
(2 rows)
restaurant_service=#
```

Рис. 2. Проверка заказов официанта до вызова процедуры 1

Перед вызовом процедуры был выполнен обычный запрос к таблице customer_order. На рисунке видно, что у официанта с идентификатором 1 есть заказы за дату 2026-03-20.

```

restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# CALL pr_lr5_waiter_orders_by_date(
restaurant_service(## 1,
restaurant_service(## DATE '2026-03-20',
restaurant_service(## 'cur_waiter_orders'
restaurant_service(## );
p_result
-----
cur_waiter_orders
(1 row)

restaurant_service=#
restaurant_service=# FETCH ALL FROM cur_waiter_orders;
 id_order | order_date | order_time | id_restaurant_table | id_customer | status | wishes | markup | total_price
-----
1 | 2026-03-20 | 18:35:00 | 2 | 1 | принят | No onion | 40.00 | 980.00
2 | 2026-03-20 | 19:05:00 | 1 | 2 | готовится |  | 35.00 | 735.00
(2 rows)

restaurant_service=#
restaurant_service=# COMMIT;
COMMIT
restaurant_service=#

```

Рис. 3. Вызов процедуры 1 и получение результата через курсор

После вызова процедуры был выполнен оператор `FETCH ALL FROM cur_waiter_orders`. В результате были получены сведения о заказах заданного официанта за заданную дату. Следовательно, процедура работает корректно.

Процедура 2. Расчет стоимости заданного заказа

Вторая процедура рассчитывает стоимость заданного заказа на основе стоимости ингредиентов блюд, входящих в заказ. Итоговая стоимость рассчитывается с учетом наценки 40% и записывается в таблицу `customer_order`.

```

restaurant_service=# CREATE OR REPLACE PROCEDURE pr_lr5_calculate_order_cost(
restaurant_service(## IN p_id_order INTEGER,
restaurant_service(## INOUT p_total_cost NUMERIC DEFAULT 0
restaurant_service(## )
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# DECLARE
restaurant_service=# v_ingredient_cost NUMERIC(10, 2);
restaurant_service=# BEGIN
restaurant_service=# IF NOT EXISTS (
restaurant_service=# SELECT 1
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_order = p_id_order
restaurant_service=# ) THEN
restaurant_service=# RAISE EXCEPTION 'Заказ с ID % не найден', p_id_order;
restaurant_service=# END IF;
restaurant_service=#
restaurant_service=# SELECT ROUND(
restaurant_service=# COALESCE(SUM(oi.quantity * di.ingredient_amount * i.unit_price), 0),
restaurant_service=# 2
restaurant_service=# )
restaurant_service=# INTO v_ingredient_cost
restaurant_service=# FROM order_item oi
restaurant_service=# JOIN dish_ingredient di
restaurant_service=# ON di.id_dish = oi.id_dish
restaurant_service=# JOIN ingredient i
restaurant_service=# ON i.id_ingredient = di.id_ingredient
restaurant_service=# WHERE oi.id_order = p_id_order;
restaurant_service=#
restaurant_service=# p_total_cost := ROUND(v_ingredient_cost * 1.40, 2);
restaurant_service=#
restaurant_service=# UPDATE customer_order
restaurant_service=# SET
restaurant_service=# markup = 40,
restaurant_service=# total_price = p_total_cost
restaurant_service=# WHERE id_order = p_id_order;
restaurant_service=# END;
restaurant_service=# $$;
CREATE PROCEDURE
restaurant_service=#

```

Рис. 4. Код процедуры `pr_lr5_calculate_order_cost`

На рисунке показано создание процедуры расчета стоимости заказа. Процедура проверяет существование заказа, рассчитывает стоимость ингредиентов и обновляет поля `markup` и `total_price`.


```

restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# SELECT
restaurant_service=#      id_order,
restaurant_service=#      markup,
restaurant_service=#      total_price
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_order = 2;
 id_order | markup | total_price
-----+-----+-----
          2 |  35.00 |      735.00
(1 row)

restaurant_service=# █

```

Рис. 5. Проверка стоимости заказа до вызова процедуры 2

До вызова процедуры заказ имел наценку 35.00 и итоговую стоимость 735.00. Это значение используется для сравнения с результатом после выполнения процедуры.

```

restaurant_service=# CALL pr_lr5_calculate_order_cost(
restaurant_service=#      2,
restaurant_service=#      NULL
restaurant_service=# );
 p_total_cost
-----
          100.80
(1 row)

restaurant_service=# █

```

Рис. 6. Вызов процедуры 2 и получение рассчитанной стоимости

После вызова процедуры было возвращено рассчитанное значение стоимости через параметр INOUT. Это показывает, что процедура не только изменяет данные, но и возвращает результат наружу.

```

restaurant_service=# SELECT
restaurant_service=#      id_order,
restaurant_service=#      markup,
restaurant_service=#      total_price
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_order = 2;
 id_order | markup | total_price
-----+-----+-----
          2 |  40.00 |      100.80
(1 row)

restaurant_service=#
restaurant_service=# ROLLBACK;
ROLLBACK
restaurant_service=# █

```

Рис. 7. Проверка стоимости заказа после вызова процедуры 2

После выполнения процедуры наценка стала равна 40.00, а итоговая стоимость заказа была пересчитана. Изменения выполнялись внутри транзакции, после проверки была выполнена команда ROLLBACK.

Процедура 3. Повышение категории и оклада сотрудника

Третья процедура изменяет категорию сотрудника и увеличивает его оклад на 30%. Старый и новый оклады возвращаются через параметры INOUT.

```

restaurant_service=# CREATE OR REPLACE PROCEDURE pr_lr5_raise_employee_category_salary(
restaurant_service(# IN p_id_employee INTEGER,
restaurant_service(# IN p_new_category VARCHAR,
restaurant_service(# INOUT p_old_salary NUMERIC DEFAULT NULL,
restaurant_service(# INOUT p_new_salary NUMERIC DEFAULT NULL
restaurant_service(# )
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service$# DECLARE
restaurant_service$# v_old_category VARCHAR;
restaurant_service$# BEGIN
restaurant_service$# SELECT
restaurant_service$# category,
restaurant_service$# salary
restaurant_service$# INTO
restaurant_service$# v_old_category,
restaurant_service$# p_old_salary
restaurant_service$# FROM employee
restaurant_service$# WHERE id_employee = p_id_employee;
restaurant_service$# IF p_old_salary IS NULL THEN
restaurant_service$# RAISE EXCEPTION 'Сотрудник с ID % не найден', p_id_employee;
restaurant_service$# END IF;
restaurant_service$# IF p_new_category IS NULL OR length(trim(p_new_category)) = 0 THEN
restaurant_service$# RAISE EXCEPTION 'Новая категория сотрудника не может быть пустой';
restaurant_service$# END IF;
restaurant_service$# IF v_old_category = p_new_category THEN
restaurant_service$# RAISE EXCEPTION 'Категория сотрудника уже равна %', p_new_category;
restaurant_service$# END IF;
restaurant_service$# p_new_salary := ROUND(p_old_salary * 1.30, 2);
restaurant_service$# UPDATE employee
restaurant_service$# SET
restaurant_service$# category = p_new_category,
restaurant_service$# salary = p_new_salary
restaurant_service$# WHERE id_employee = p_id_employee;
restaurant_service$# END;
restaurant_service=# $$;
CREATE PROCEDURE
restaurant_service=#

```

Рис. 8. Код процедуры pr_lr5_raise_employee_category_salary

На рисунке показано создание процедуры. В процедуре выбираются текущая категория и оклад сотрудника, выполняются проверки, после чего рассчитывается новый оклад.

```

restaurant_service=# BEGIN;
BEGIN
restaurant_service==#
restaurant_service==# SELECT
restaurant_service==# id_employee,
restaurant_service==# last_name,
restaurant_service==# first_name,
restaurant_service==# category,
restaurant_service==# salary
restaurant_service==# FROM employee
restaurant_service==# WHERE id_employee = 4;
restaurant_service==#
id_employee | last_name | first_name | category | salary
-----+-----+-----+-----+-----
(1 row)
4 | Smirnova | Elena | C | 45000.00
restaurant_service==#

```

Рис. 9. Проверка данных сотрудника до вызова процедуры 3

До выполнения процедуры сотрудник с идентификатором 4 имел категорию C и оклад 45000.00.

```

restaurant_service==# CALL pr_lr5_raise_employee_category_salary(
restaurant_service==# 4,
restaurant_service==# 'B',
restaurant_service==# NULL,
restaurant_service==# NULL
restaurant_service==# );
p_old_salary | p_new_salary
-----+-----
(1 row)
45000.00 | 58500.00
restaurant_service==#

```

Рис. 10. Вызов процедуры 3 и получение старого и нового оклада

После вызова процедуры были получены старый и новый оклады: 45000.00 и 58500.00. Новый оклад соответствует увеличению на 30%.

```
restaurant_service=# SELECT
restaurant_service=#     id_employee,
restaurant_service=#     last_name,
restaurant_service=#     first_name,
restaurant_service=#     category,
restaurant_service=#     salary
restaurant_service=# FROM employee
restaurant_service=# WHERE id_employee = 4;
 id_employee | last_name | first_name | category | salary
-----+-----+-----+-----+-----
          4 | Smirnova | Elena     | B        | 58500.00
(1 row)

restaurant_service=#
restaurant_service=# ROLLBACK;
ROLLBACK
restaurant_service=#
```

Рис. 11. Проверка данных сотрудника после вызова процедуры 3

После выполнения процедуры категория сотрудника изменилась на В, а оклад увеличился до 58500.00. После проверки изменения были отменены командой ROLLBACK.

4.2. Создание таблицы аудита

Для фиксации действий, выполняемых частью триггеров, была создана служебная таблица `audit_log`. В ней сохраняются имя таблицы, тип операции, идентификатор измененной строки и описание действия.

```
restaurant_service=# CREATE TABLE IF NOT EXISTS audit_log (
restaurant_service(#   id_log INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
restaurant_service(#   action_datetime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
restaurant_service(#   table_name VARCHAR(80) NOT NULL,
restaurant_service(#   operation VARCHAR(20) NOT NULL,
restaurant_service(#   row_id INTEGER,
restaurant_service(#   description TEXT
restaurant_service(# );
restaurant_service=# CREATE TABLE
restaurant_service=#
```

Рис. 12. Создание таблицы `audit_log`

Таблица аудита используется в триггерах, которые фиксируют операции с позициями заказа, закупками и изменением доступности блюд.

4.3. Создание и проверка триггеров

В рамках лабораторной работы были созданы 10 триггеров. Их назначение представлено в таблице 1.

Таблица 1. Разработанные триггеры

№	Триггер	Назначение
1	<code>trg_customer_order_active_table</code>	Запрещает создание второго активного заказа на один и тот же стол.
2	<code>trg_customer_order_status_transition</code>	Проверяет допустимость переходов статуса заказа.

3	trg_order_item_calculate_price	Автоматически рассчитывает себестоимость и стоимость позиции заказа.
4	trg_order_item_check_stock	Проверяет достаточность ингредиентов для выбранного блюда.
5	trg_order_item_recalculate_total	Пересчитывает итоговую сумму заказа после изменения позиций заказа.
6	trg_order_item_audit	Записывает операции с позициями заказа в таблицу аудита.
7	trg_purchase_validate	Проверяет корректность данных закупки.
8	trg_purchase_update_stock	Автоматически обновляет остаток ингредиента после закупки.
9	trg_ingredient_update_dish_availability	Обновляет доступность блюд после изменения остатка ингредиента.
10	trg_cooking_assignment_validate	Проверяет корректность назначения приготовления блюда.

Триггер 1. Запрет второго активного заказа на один стол

Первый триггер проверяет, есть ли уже активный заказ на выбранном столе. Активным считается заказ, статус которого не равен **закрыт**.

```
restaurant_service=# CREATE OR REPLACE FUNCTION fn_customer_order_active_table()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# BEGIN
restaurant_service=#     IF NEW.status <> 'закрыт'
restaurant_service=#         AND EXISTS (
restaurant_service=#             SELECT 1
restaurant_service=#             FROM customer_order
restaurant_service=#             WHERE id_restaurant_table = NEW.id_restaurant_table
restaurant_service=#             AND status <> 'закрыт'
restaurant_service=#             AND id_order <> NEW.id_order
restaurant_service=#         )
restaurant_service=#     THEN
restaurant_service=#         RAISE EXCEPTION 'У стола ID % уже есть активный заказ', NEW.id_restaurant_table;
restaurant_service=#     END IF;
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
restaurant_service=# CREATE TRIGGER
restaurant_service=#
```

Рис. 13. Код триггерной функции для триггера 1

Функция проверяет таблицу **customer_order** и генерирует исключение, если для того же стола уже существует активный заказ.

```
restaurant_service=# DROP TRIGGER IF EXISTS trg_customer_order_active_table ON customer_order;
NOTICE: trigger "trg_customer_order_active_table" for relation "customer_order" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_customer_order_active_table
restaurant_service=# BEFORE INSERT OR UPDATE ON customer_order
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_customer_order_active_table();
CREATE TRIGGER
restaurant_service=#
```

Рис. 14. Создание триггера 1

Триггер создан перед операциями **INSERT** и **UPDATE**. Поэтому проверка выполняется до записи данных в таблицу.

```

restaurant_service=# SELECT
restaurant_service=#     id_order,
restaurant_service=#     id_restaurant_table,
restaurant_service=#     status
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_restaurant_table = 1
restaurant_service=# ORDER BY id_order;
 id_order | id_restaurant_table | status
-----+-----+-----
      2 |          1 | готовится
(1 row)

restaurant_service=#

```

Рис. 15. Проверка активного заказа на столе до срабатывания триггера 1

Перед проверкой был найден активный заказ на столе с идентификатором 1. Поэтому новая вставка активного заказа на этот же стол должна быть заблокирована.

```

restaurant_service=# INSERT INTO customer_order (
restaurant_service=#     id_order,
restaurant_service=#     id_restaurant_table,
restaurant_service=#     id_waiter,
restaurant_service=#     id_customer,
restaurant_service=#     id_work_shift,
restaurant_service=#     order_datetime,
restaurant_service=#     status,
restaurant_service=#     wishes,
restaurant_service=#     markup,
restaurant_service=#     total_price
restaurant_service=# )
restaurant_service=# VALUES (
restaurant_service=#     (SELECT COALESCE(MAX(id_order), 0) + 1 FROM customer_order),
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     CURRENT_TIMESTAMP,
restaurant_service=#     'принят',
restaurant_service=#     'Проверка триггера активного заказа',
restaurant_service=#     40,
restaurant_service=#     0
restaurant_service=# );
ERROR:  У стола ID 1 уже есть активный заказ
CONTEXT:  PL/pgSQL function fn_customer_order_active_table() line 12 at RAISE
restaurant_service=#

```

Рис. 16. Вызов действия, запускающего триггер 1

Попытка создать второй активный заказ завершилась ошибкой. Это подтверждает, что триггер 1 работает корректно.

Триггер 2. Проверка допустимого перехода статуса заказа

Второй триггер контролирует последовательность изменения статуса заказа. Допустимая цепочка переходов: принят -> готовится -> подан -> закрыт.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_customer_order_status_transition()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# BEGIN
restaurant_service=#     IF TG_OP = 'INSERT' THEN
restaurant_service=#         RETURN NEW;
restaurant_service=#     END IF;
restaurant_service=#     IF OLD.status = NEW.status THEN
restaurant_service=#         RETURN NEW;
restaurant_service=#     END IF;
restaurant_service=#     IF OLD.status = 'принят' AND NEW.status = 'готовится' THEN
restaurant_service=#         RETURN NEW;
restaurant_service=#     END IF;
restaurant_service=#     IF OLD.status = 'готовится' AND NEW.status = 'подан' THEN
restaurant_service=#         RETURN NEW;
restaurant_service=#     END IF;
restaurant_service=#     IF OLD.status = 'подан' AND NEW.status = 'закрыт' THEN
restaurant_service=#         RETURN NEW;
restaurant_service=#     END IF;
restaurant_service=#     RAISE EXCEPTION 'Недопустимый переход статуса заказа: % -> %', OLD.status, NEW.status;
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#

```

Рис. 17. Код триггерной функции для триггера 2

Функция сравнивает старый и новый статус заказа. Если переход не входит в список допустимых, операция блокируется.

```
restaurant_service=# DROP TRIGGER IF EXISTS trg_customer_order_status_transition ON customer_order;
NOTICE: trigger "trg_customer_order_status_transition" for relation "customer_order" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_customer_order_status_transition
restaurant_service=# BEFORE UPDATE OF status ON customer_order
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_customer_order_status_transition();
CREATE TRIGGER
restaurant_service=#
```

Рис. 18. Создание триггера 2

Триггер создан перед обновлением поля `status` таблицы `customer_order`.

```
restaurant_service=# SELECT
restaurant_service=#     id_order,
restaurant_service=#     status
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_order = 1;
 id_order | status
-----+-----
         1 | принят
(1 row)

restaurant_service=#
```

Рис. 19. Проверка статуса заказа до срабатывания триггера 2

Перед проверкой заказ находился в статусе `принят`.

```
restaurant_service=# UPDATE customer_order
restaurant_service=# SET status = 'подан'
restaurant_service=# WHERE id_order = 1;
ERROR: Недопустимый переход статуса заказа: принят -> подан
CONTEXT: PL/pgSQL function fn_customer_order_status_transition() line 23 at RAISE
restaurant_service=#
```

Рис. 20. Вызов действия, запускающего триггер 2

Переход `принят -> подан` был запрещен, потому что перед статусом `подан` заказ должен перейти в статус `готовится`.

```
restaurant_service=# UPDATE customer_order
restaurant_service=# SET status = 'готовится'
restaurant_service=# WHERE id_order = 1;
UPDATE 1
restaurant_service=#
restaurant_service=# SELECT
restaurant_service=#     id_order,
restaurant_service=#     status
restaurant_service=# FROM customer_order
restaurant_service=# WHERE id_order = 1;
 id_order | status
-----+-----
         1 | готовится
(1 row)

restaurant_service=#
```

Рис. 21. Проверка корректного перехода статуса заказа

Корректный переход `принят -> готовится` был выполнен успешно. Это подтверждает, что триггер блокирует только неправильные переходы.

Триггер 3. Автоматический расчет стоимости позиции заказа

Третий триггер автоматически рассчитывает поля `cost_price` и `total_price` при добавлении или изменении позиции заказа.

```
restaurant_service=# CREATE OR REPLACE FUNCTION fn_order_item_calculate_price()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# DECLARE
restaurant_service=#     v_cost_price NUMERIC(10,2);
restaurant_service=# BEGIN
restaurant_service=#     SELECT ROUND(SUM(di.ingredient_amount * i.unit_price), 2)
restaurant_service=#     INTO v_cost_price
restaurant_service=#     FROM dish_ingredient di
restaurant_service=#     JOIN ingredient i
restaurant_service=#     ON i.id_ingredient = di.id_ingredient
restaurant_service=#     WHERE di.id_dish = NEW.id_dish;
restaurant_service=#     IF v_cost_price IS NULL OR v_cost_price <= 0 THEN
restaurant_service=#         RAISE EXCEPTION 'Невозможно рассчитать себестоимость блюда ID %', NEW.id_dish;
restaurant_service=#     END IF;
restaurant_service=#     NEW.cost_price := v_cost_price;
restaurant_service=#     NEW.total_price := ROUND(v_cost_price * NEW.quantity, 2);
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#
```

Рис. 22. Код триггерной функции для триггера 3

Функция суммирует стоимость ингредиентов блюда и умножает ее на количество заказанных порций.

```
restaurant_service=# DROP TRIGGER IF EXISTS trg_order_item_calculate_price ON order_item;
NOTICE: trigger "trg_order_item_calculate_price" for relation "order_item" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_order_item_calculate_price
restaurant_service=# BEFORE INSERT OR UPDATE OF id_dish, quantity ON order_item
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_order_item_calculate_price();
CREATE TRIGGER
restaurant_service=#
```

Рис. 23. Создание триггера 3

Триггер создан перед вставкой и обновлением полей `id_dish` и `quantity`. Поэтому стоимость позиции рассчитывается автоматически до записи строки.

```
restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# INSERT INTO order_item (
restaurant_service=#     id_order_item,
restaurant_service=#     id_order,
restaurant_service=#     id_dish,
restaurant_service=#     quantity,
restaurant_service=#     cost_price,
restaurant_service=#     total_price,
restaurant_service=#     wishes
restaurant_service=# )
restaurant_service=# VALUES (
restaurant_service=#     (SELECT COALESCE(MAX(id_order_item), 0) + 1 FROM order_item),
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     2,
restaurant_service=#     0,
restaurant_service=#     0,
restaurant_service=#     'Проверка расчета цены'
restaurant_service=# );
INSERT 0 1
restaurant_service=#
```

Рис. 24. Вызов действия, запускающего триггер 3

Для проверки была добавлена новая позиция заказа с нулевыми значениями `cost_price` и `total_price`.

```

restaurant_service=# SELECT
restaurant_service=#     id_order_item,
restaurant_service=#     id_order,
restaurant_service=#     id_dish,
restaurant_service=#     quantity,
restaurant_service=#     cost_price,
restaurant_service=#     total_price,
restaurant_service=#     wishes
restaurant_service=# FROM order_item
restaurant_service=# WHERE wishes = 'Проверка расчета цены';
 id_order_item | id_order | id_dish | quantity | cost_price | total_price | wishes
-----
(1 row)
 4 | 1 | 1 | 2 | 147.00 | 294.00 | Проверка расчета цены

restaurant_service=#
restaurant_service=# ROLLBACK;
restaurant_service=#

```

Рис. 25. Проверка данных после срабатывания триггера 3

После вставки стоимость была рассчитана автоматически: `cost_price` = 147.00, `total_price` = 294.00.

Триггер 4. Проверка достаточности ингредиентов

Четвертый триггер проверяет, достаточно ли ингредиентов на складе для добавления выбранного блюда в заказ.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_order_item_check_stock()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# DECLARE
restaurant_service=#     v_ingredient_name VARCHAR(120);
restaurant_service=# BEGIN
restaurant_service=#     SELECT i.ingredient_name
restaurant_service=#     INTO v_ingredient_name
restaurant_service=#     FROM dish_ingredient di
restaurant_service=#     JOIN ingredient i
restaurant_service=#         ON i.id_ingredient = di.id_ingredient
restaurant_service=#     WHERE di.id_dish = NEW.id_dish
restaurant_service=#     AND i.stock_quantity < di.ingredient_amount * NEW.quantity
restaurant_service=#     LIMIT 1;
restaurant_service=#     IF v_ingredient_name IS NOT NULL THEN
restaurant_service=#         RAISE EXCEPTION 'Недостаточно ингредиента "%" для блюда ID %',
restaurant_service=#             v_ingredient_name,
restaurant_service=#             NEW.id_dish;
restaurant_service=#     END IF;
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
restaurant_service=# CREATE FUNCTION
restaurant_service=#

```

Рис. 26. Код триггерной функции для триггера 4

Функция ищет ингредиент, остаток которого меньше, чем требуется для выбранного блюда с учетом количества порций.

```

restaurant_service=# DROP TRIGGER IF EXISTS trg_order_item_check_stock ON order_item;
NOTICE: trigger "trg_order_item_check_stock" for relation "order_item" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_order_item_check_stock
restaurant_service=# BEFORE INSERT OR UPDATE OF id_dish, quantity ON order_item
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_order_item_check_stock();
CREATE TRIGGER
restaurant_service=#

```

Рис. 27. Создание триггера 4

Триггер создан перед вставкой и обновлением полей `id_dish` и `quantity` в таблице `order_item`.


```

restaurant_service=# CREATE TRIGGER trg_order_item_check_stock
restaurant_service=# BEFORE INSERT OR UPDATE OF id_dish, quantity ON order_item
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_order_item_check_stock();
CREATE TRIGGER
restaurant_service=# SELECT
restaurant_service=#     i.id_ingredient,
restaurant_service=#     i.ingredient_name,
restaurant_service=#     i.stock_quantity,
restaurant_service=#     di.id_dish,
restaurant_service=#     di.ingredient_amount
restaurant_service=# FROM ingredient i
restaurant_service=# JOIN dish_ingredient di
restaurant_service=#     ON di.id_ingredient = i.id_ingredient
restaurant_service=# WHERE di.id_dish = 1
restaurant_service=# ORDER BY i.id_ingredient;
 id_ingredient | ingredient_name | stock_quantity | id_dish | ingredient_amount
-----
1 | Tomato         | 15.000         | 1       | 0.150
2 | Chicken        | 10.000         | 1       | 0.200
3 | Parmesan       | 3.000          | 1       | 0.050
(3 rows)
restaurant_service=#

```

Рис. 28. Попытка добавления позиции заказа при нехватке ингредиентов

Для проверки была выполнена попытка добавить слишком большое количество блюда.

```

restaurant_service=# INSERT INTO order_item (
restaurant_service=#     id_order_item,
restaurant_service=#     id_order,
restaurant_service=#     id_dish,
restaurant_service=#     quantity,
restaurant_service=#     cost_price,
restaurant_service=#     total_price,
restaurant_service=#     wishes
restaurant_service=# )
restaurant_service=# VALUES (
restaurant_service=#     (SELECT COALESCE(MAX(id_order_item), 0) + 1 FROM order_item),
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     100000,
restaurant_service=#     0,
restaurant_service=#     0,
restaurant_service=#     'Проверка нехватки ингредиентов'
restaurant_service=# );
ERROR: Недостаточно ингредиента "Tomato" для блюда ID 1
CONTEXT: PL/pgSQL function fn_order_item_check_stock() line 15 at RAISE
restaurant_service=#

```

Рис. 29. Сообщение об ошибке при нехватке ингредиентов

Операция была заблокирована, так как недостаточно ингредиента Tomato. Это подтверждает работу триггера 4.

Триггер 5. Автоматический пересчет итоговой суммы заказа

Пятый триггер пересчитывает итоговую сумму заказа после добавления, изменения или удаления позиции заказа.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_order_item_recalculate_total()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# DECLARE
restaurant_service=#     v_order_id INTEGER;
restaurant_service=# BEGIN
restaurant_service=#     IF TG_OP = 'DELETE' THEN
restaurant_service=#         v_order_id := OLD.id_order;
restaurant_service=#     ELSE
restaurant_service=#         v_order_id := NEW.id_order;
restaurant_service=#     END IF;
restaurant_service=#     UPDATE customer_order
restaurant_service=#     SET total_price = COALESCE(
restaurant_service=#         SELECT ROUND(SUM(oi.total_price) * (1 + customer_order.markup / 100), 2)
restaurant_service=#         FROM order_item oi
restaurant_service=#         WHERE oi.id_order = v_order_id
restaurant_service=#     ), 0)
restaurant_service=#     WHERE id_order = v_order_id;
restaurant_service=#     RETURN COALESCE(NEW, OLD);
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#

```

Рис. 30. Код триггерной функции для триггера 5

Функция определяет заказ, к которому относится измененная позиция, суммирует все позиции и применяет наценку из таблицы `customer_order`.

```
restaurant_service=# DROP TRIGGER IF EXISTS trg_order_item_recalculate_total ON order_item;
NOTICE: trigger "trg_order_item_recalculate_total" for relation "order_item" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_order_item_recalculate_total
restaurant_service=# AFTER INSERT OR UPDATE OR DELETE ON order_item
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_order_item_recalculate_total();
CREATE TRIGGER
restaurant_service=#
```

Рис. 31. Создание триггера 5

Триггер создан после операций `INSERT`, `UPDATE` и `DELETE` в таблице `order_item`.

```
restaurant_service=# BEGIN;
BEGIN
restaurant_service==#
restaurant_service==# SELECT
restaurant_service==#     id_order,
restaurant_service==#     markup,
restaurant_service==#     total_price
restaurant_service==# FROM customer_order
restaurant_service==# WHERE id_order = 1;
 id_order | markup | total_price
-----+-----+-----
      1  |  40.00 |   980.00
(1 row)
restaurant_service==#
```

Рис. 32. Проверка суммы заказа до срабатывания триггера 5

Перед проверкой сумма заказа составляла 980.00.

```
restaurant_service==# INSERT INTO order_item (
restaurant_service==#     id_order_item,
restaurant_service==#     id_order,
restaurant_service==#     id_dish,
restaurant_service==#     quantity,
restaurant_service==#     cost_price,
restaurant_service==#     total_price,
restaurant_service==#     wishes
restaurant_service==# )
restaurant_service==# VALUES (
restaurant_service==#     (SELECT COALESCE(MAX(id_order_item), 0) + 1 FROM order_item),
restaurant_service==#     1,
restaurant_service==#     1,
restaurant_service==#     1,
restaurant_service==#     0,
restaurant_service==#     0,
restaurant_service==#     'Проверка пересчета суммы заказа'
restaurant_service==# );
INSERT 0 1
restaurant_service==#
```

Рис. 33. Вызов действия, запускающего триггер 5

Для запуска триггера была добавлена новая позиция заказа.

```
restaurant_service==# SELECT
restaurant_service==#     id_order,
restaurant_service==#     markup,
restaurant_service==#     total_price
restaurant_service==# FROM customer_order
restaurant_service==# WHERE id_order = 1;
 id_order | markup | total_price
-----+-----+-----
      1  |  40.00 |  1437.80
(1 row)

restaurant_service==#
restaurant_service==# ROLLBACK;
ROLLBACK
restaurant_service=#
```

Рис. 34. Проверка суммы заказа после срабатывания триггера 5

После добавления позиции итоговая сумма заказа изменилась на 1437.80. Это подтверждает автоматический пересчет суммы заказа.

Триггер 6. Аудит операций с позициями заказа

Шестой триггер записывает в таблицу audit_log информацию об операциях с позициями заказа.

```
restaurant_service=# CREATE OR REPLACE FUNCTION fn_order_item_audit()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service$# BEGIN
restaurant_service$#     IF TG_OP = 'INSERT' THEN
restaurant_service$#         INSERT INTO audit_log (
restaurant_service$#             table_name,
restaurant_service$#             operation,
restaurant_service$#             row_id,
restaurant_service$#             description
restaurant_service$#         )
restaurant_service$#         VALUES (
restaurant_service$#             'order_item',
restaurant_service$#             'INSERT',
restaurant_service$#             NEW.id_order_item,
restaurant_service$#             'Добавлена позиция заказа'
restaurant_service$#         );
restaurant_service$#     RETURN NEW;
restaurant_service$#     END IF;
restaurant_service$#     IF TG_OP = 'UPDATE' THEN
restaurant_service$#         INSERT INTO audit_log (
restaurant_service$#             table_name,
restaurant_service$#             operation,
restaurant_service$#             row_id,
restaurant_service$#             description
restaurant_service$#         )
restaurant_service$#         VALUES (
restaurant_service$#             'order_item',
restaurant_service$#             'UPDATE',
restaurant_service$#             NEW.id_order_item,
restaurant_service$#             'Изменена позиция заказа'
restaurant_service$#         );
restaurant_service$#     RETURN NEW;
restaurant_service$#     END IF;
restaurant_service$#     IF TG_OP = 'DELETE' THEN
restaurant_service$#         INSERT INTO audit_log (
restaurant_service$#             table_name,
restaurant_service$#             operation,
restaurant_service$#             row_id,
restaurant_service$#             description
restaurant_service$#         )
restaurant_service$#         VALUES (
restaurant_service$#             'order_item',
restaurant_service$#             'DELETE',
restaurant_service$#             OLD.id_order_item,
restaurant_service$#             'Удалена позиция заказа'
restaurant_service$#         );
restaurant_service$#     RETURN OLD;
restaurant_service$#     END IF;
restaurant_service$#     RETURN NULL;
restaurant_service$# END;
restaurant_service$# $$;
CREATE FUNCTION
restaurant_service=#
```

Рис. 35. Код триггерной функции для триггера 6

Функция обрабатывает операции INSERT, UPDATE и DELETE и для каждой операции создает запись в таблице аудита.

```

restaurant_service=# DROP TRIGGER IF EXISTS trg_order_item_audit ON order_item;
NOTICE: trigger "trg_order_item_audit" for relation "order_item" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_order_item_audit
restaurant_service=# AFTER INSERT OR UPDATE OR DELETE ON order_item
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_order_item_audit();
CREATE TRIGGER
restaurant_service=# █

```

Рис. 36. Создание триггера 6

Триггер создан после вставки, обновления и удаления строк в таблице `order_item`.

```

restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# SELECT
restaurant_service=#     id_log,
restaurant_service=#     table_name,
restaurant_service=#     operation,
restaurant_service=#     row_id,
restaurant_service=#     description
restaurant_service=# FROM audit_log
restaurant_service=# ORDER BY id_log DESC
restaurant_service=# LIMIT 5;
id_log | table_name | operation | row_id | description
-----+-----+-----+-----+-----
(0 rows)
restaurant_service=# █

```

Рис. 37. Проверка таблицы аудита до срабатывания триггера 6

Перед проверкой таблица аудита была пустой.

```

restaurant_service=# INSERT INTO order_item (
restaurant_service=#     id_order_item,
restaurant_service=#     id_order,
restaurant_service=#     id_dish,
restaurant_service=#     quantity,
restaurant_service=#     cost_price,
restaurant_service=#     total_price,
restaurant_service=#     wishes
restaurant_service=# )
restaurant_service=# VALUES (
restaurant_service=#     (SELECT COALESCE(MAX(id_order_item), 0) + 1 FROM order_item),
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     0,
restaurant_service=#     0,
restaurant_service=#     'Проверка аудита позиции заказа'
restaurant_service=# );
INSERT 0 1
restaurant_service=# █

```

Рис. 38. Вызов действия, запускающего триггер 6

Для запуска триггера была добавлена новая позиция заказа.


```

restaurant_service==# SELECT
restaurant_service==#     id_log,
restaurant_service==#     table_name,
restaurant_service==#     operation,
restaurant_service==#     row_id,
restaurant_service==#     description
restaurant_service==# FROM audit_log
restaurant_service==# ORDER BY id_log DESC
restaurant_service==# LIMIT 5;
id_log | table_name | operation | row_id | description
-----+-----+-----+-----+-----
1 | order_item | INSERT | 4 | Добавлена позиция заказа
(1 row)

restaurant_service==#
restaurant_service==# ROLLBACK;
ROLLBACK
restaurant_service=# █

```

Рис. 39. Проверка таблицы аудита после срабатывания триггера 6

После вставки позиции в таблице аудита появилась запись с операцией INSERT. Это подтверждает работу триггера аудита.

Триггер 7. Проверка корректности закупки

Седьмой триггер проверяет корректность данных закупки: объем, цену, остаток партии и срок годности.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_purchase_validate()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# BEGIN
restaurant_service=#     IF NEW.purchase_volume <= 0 THEN
restaurant_service=#         RAISE EXCEPTION 'Объем закупки должен быть больше 0';
restaurant_service=#     END IF;
restaurant_service=#     IF NEW.purchase_price <= 0 THEN
restaurant_service=#         RAISE EXCEPTION 'Цена закупки должна быть больше 0';
restaurant_service=#     END IF;
restaurant_service=#     IF NEW.batch_balance < 0 THEN
restaurant_service=#         RAISE EXCEPTION 'Остаток партии не может быть отрицательным';
restaurant_service=#     END IF;
restaurant_service=#     IF NEW.batch_balance > NEW.purchase_volume THEN
restaurant_service=#         RAISE EXCEPTION 'Остаток партии не может быть больше объема закупки';
restaurant_service=#     END IF;
restaurant_service=#     IF NEW.expiration_date < NEW.purchase_date THEN
restaurant_service=#         RAISE EXCEPTION 'Срок годности не может быть раньше даты закупки';
restaurant_service=#     END IF;
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=# █

```

Рис. 40. Код триггерной функции для триггера 7

Функция запрещает некорректные закупки: с нулевым или отрицательным объемом, некорректной ценой, отрицательным остатком партии или неправильным сроком годности.

```

restaurant_service=# DROP TRIGGER IF EXISTS trg_purchase_validate ON purchase;
NOTICE: trigger "trg_purchase_validate" for relation "purchase" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_purchase_validate
restaurant_service=# BEFORE INSERT OR UPDATE ON purchase
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_purchase_validate();
CREATE TRIGGER
restaurant_service=# █

```

Рис. 41. Создание триггера 7

Триггер создан перед операциями INSERT и UPDATE в таблице purchase.

```
restaurant_service=# INSERT INTO purchase (
restaurant_service#      id_purchase,
restaurant_service#      id_ingredient,
restaurant_service#      id_supplier,
restaurant_service#      purchase_date,
restaurant_service#      purchase_volume,
restaurant_service#      purchase_price,
restaurant_service#      batch_balance,
restaurant_service#      expiration_date
restaurant_service# )
restaurant_service=# VALUES (
restaurant_service#      (SELECT COALESCE(MAX(id_purchase), 0) + 1 FROM purchase),
restaurant_service#      1,
restaurant_service#      1,
restaurant_service#      CURRENT_DATE,
restaurant_service#      5,
restaurant_service#      750,
restaurant_service#      5,
restaurant_service#      CURRENT_DATE - INTERVAL '1 day'
restaurant_service# );
ERROR: Срок годности не может быть раньше даты закупки
CONTEXT: PL/pgSQL function fn_purchase_validate() line 20 at RAISE
restaurant_service=#
```

Рис. 42. Вызов действия, запускающего триггер 7

Попытка создать закупку со сроком годности раньше даты закупки была заблокирована.

Триггер 8. Автоматическое обновление остатка ингредиента после закупки

Восьмой триггер автоматически обновляет данные ингредиента после регистрации новой закупки.

```
restaurant_service=# CREATE OR REPLACE FUNCTION fn_purchase_update_stock()
restaurant_service# RETURNS trigger
restaurant_service# LANGUAGE plpgsql
restaurant_service# AS $$
restaurant_service# BEGIN
restaurant_service#     UPDATE ingredient
restaurant_service#     SET
restaurant_service#         stock_quantity = stock_quantity + NEW.purchase_volume,
restaurant_service#         unit_price = ROUND(NEW.purchase_price / NEW.purchase_volume, 2),
restaurant_service#         storage_life = GREATEST(storage_life, NEW.expiration_date)
restaurant_service#     WHERE id_ingredient = NEW.id_ingredient;
restaurant_service#
restaurant_service#     INSERT INTO audit_log (
restaurant_service#         table_name,
restaurant_service#         operation,
restaurant_service#         row_id,
restaurant_service#         description
restaurant_service#     )
restaurant_service#     VALUES (
restaurant_service#         'purchase',
restaurant_service#         'INSERT',
restaurant_service#         NEW.id_purchase,
restaurant_service#         'После закупки автоматически увеличен остаток ингредиента'
restaurant_service#     );
restaurant_service#     RETURN NEW;
restaurant_service# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#
```

Рис. 43. Код триггерной функции для триггера 8

Функция увеличивает остаток ингредиента на объем закупки, пересчитывает цену за единицу и обновляет срок хранения.

```
restaurant_service=# DROP TRIGGER IF EXISTS trg_purchase_update_stock ON purchase;
NOTICE: trigger "trg_purchase_update_stock" for relation "purchase" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_purchase_update_stock
restaurant_service# AFTER INSERT ON purchase
restaurant_service# FOR EACH ROW
restaurant_service# EXECUTE FUNCTION fn_purchase_update_stock();
CREATE TRIGGER
restaurant_service=#
```

Рис. 44. Создание триггера 8

Триггер создан после операции INSERT в таблице purchase.

```
restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# SELECT
restaurant_service=#     id_ingredient,
restaurant_service=#     ingredient_name,
restaurant_service=#     unit_price,
restaurant_service=#     stock_quantity,
restaurant_service=#     storage_life
restaurant_service=# FROM ingredient
restaurant_service=# WHERE id_ingredient = 1;
 id_ingredient | ingredient_name | unit_price | stock_quantity | storage_life
-----+-----+-----+-----+-----
(1 row)
1 | Tomato | 120.00 | 15.000 | 2026-03-30
restaurant_service=# █
```

Рис. 45. Проверка ингредиента до срабатывания триггера 8

Перед закупкой у ингредиента Tomato цена была равна 120.00, остаток — 15.000, срок хранения — 2026-03-30.

```
restaurant_service=# INSERT INTO purchase (
restaurant_service=#     id_purchase,
restaurant_service=#     id_ingredient,
restaurant_service=#     id_supplier,
restaurant_service=#     purchase_date,
restaurant_service=#     purchase_volume,
restaurant_service=#     purchase_price,
restaurant_service=#     batch_balance,
restaurant_service=#     expiration_date
restaurant_service=# )
restaurant_service=# VALUES (
restaurant_service=#     (SELECT COALESCE(MAX(id_purchase), 0) + 1 FROM purchase),
restaurant_service=#     1,
restaurant_service=#     1,
restaurant_service=#     CURRENT_DATE,
restaurant_service=#     5,
restaurant_service=#     750,
restaurant_service=#     5,
restaurant_service=#     CURRENT_DATE + INTERVAL '30 day'
restaurant_service=# );
INSERT 0 1
restaurant_service=# █
```

Рис. 46. Вызов действия, запускающего триггер 8

Для проверки была добавлена закупка ингредиента Tomato объемом 5 по цене 750.

```
restaurant_service=# SELECT
restaurant_service=#     id_ingredient,
restaurant_service=#     ingredient_name,
restaurant_service=#     unit_price,
restaurant_service=#     stock_quantity,
restaurant_service=#     storage_life
restaurant_service=# FROM ingredient
restaurant_service=# WHERE id_ingredient = 1;
 id_ingredient | ingredient_name | unit_price | stock_quantity | storage_life
-----+-----+-----+-----+-----
(1 row)
1 | Tomato | 150.00 | 20.000 | 2026-06-02
restaurant_service=#
restaurant_service=# ROLLBACK;
ROLLBACK
restaurant_service=# █
```

Рис. 47. Проверка ингредиента после срабатывания триггера 8

После срабатывания триггера остаток увеличился до 20.000, цена стала 150.00, а срок хранения был обновлен.

Триггер 9. Обновление доступности блюд после изменения остатка ингредиента

Девятый триггер обновляет доступность блюд при изменении остатка ингредиента.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_ingredient_update_dish_availability()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# BEGIN
restaurant_service=#     UPDATE dish d
restaurant_service=#     SET is_available = NOT EXISTS (
restaurant_service=#         SELECT 1
restaurant_service=#         FROM dish_ingredient di
restaurant_service=#         JOIN ingredient i
restaurant_service=#             ON i.id_ingredient = di.id_ingredient
restaurant_service=#         WHERE di.id_dish = d.id_dish
restaurant_service=#             AND i.stock_quantity < di.ingredient_amount
restaurant_service=#     )
restaurant_service=#     WHERE d.id_dish IN (
restaurant_service=#         SELECT id_dish
restaurant_service=#         FROM dish_ingredient
restaurant_service=#         WHERE id_ingredient = NEW.id_ingredient
restaurant_service=#     );
restaurant_service=#     INSERT INTO audit_log (
restaurant_service=#         table_name,
restaurant_service=#         operation,
restaurant_service=#         row_id,
restaurant_service=#         description
restaurant_service=#     )
restaurant_service=#     VALUES (
restaurant_service=#         'ingredient',
restaurant_service=#         'UPDATE',
restaurant_service=#         NEW.id_ingredient,
restaurant_service=#         'После изменения остатка ингредиента обновлена доступность блюд'
restaurant_service=#     );
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#

```

Рис. 48. Код триггерной функции для триггера 9

Функция проверяет все блюда, в которых используется измененный ингредиент. Если ингредиента недостаточно, поле `is_available` становится равным `false`.

```

restaurant_service=# DROP TRIGGER IF EXISTS trg_ingredient_update_dish_availability ON ingredient;
NOTICE: trigger "trg_ingredient_update_dish_availability" for relation "ingredient" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_ingredient_update_dish_availability
restaurant_service=# AFTER UPDATE OF stock_quantity ON ingredient
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_ingredient_update_dish_availability();
CREATE TRIGGER
restaurant_service=#

```

Рис. 49. Создание триггера 9

Триггер создан после обновления поля `stock_quantity` таблицы `ingredient`.

```

restaurant_service=# BEGIN;
BEGIN
restaurant_service=# SELECT
restaurant_service=#     d.id_dish,
restaurant_service=#     d.dish_name,
restaurant_service=#     d.is_available,
restaurant_service=#     i.id_ingredient,
restaurant_service=#     i.ingredient_name,
restaurant_service=#     i.stock_quantity,
restaurant_service=#     di.ingredient_amount
restaurant_service=# FROM dish d
restaurant_service=# JOIN dish_ingredient di
restaurant_service=#     ON di.id_dish = d.id_dish
restaurant_service=# JOIN ingredient i
restaurant_service=#     ON i.id_ingredient = di.id_ingredient
restaurant_service=# WHERE i.id_ingredient = 1
restaurant_service=# ORDER BY d.id_dish;
 id_dish | dish_name | is_available | id_ingredient | ingredient_name | stock_quantity | ingredient_amount
-----
(1 row)
 1 | Caesar Salad | t           | 1 | Tomato         | 15.000         | 0.150
restaurant_service=#

```

Рис. 50. Проверка доступности блюд до срабатывания триггера 9

Перед проверкой блюдо **Caesar Salad**, использующее ингредиент **Tomato**, было доступно.

```

restaurant_service=# BEGIN;
BEGIN
restaurant_service=#
restaurant_service=# UPDATE ingredient
restaurant_service=# SET storage_life = CURRENT_DATE + INTERVAL '30 day'
restaurant_service=# WHERE id_ingredient = 1;
UPDATE 1
restaurant_service=#
restaurant_service=# UPDATE ingredient
restaurant_service=# SET stock_quantity = 0
restaurant_service=# WHERE id_ingredient = 1;
UPDATE 1
restaurant_service=# █

```

Рис. 51. Изменение остатка ингредиента для запуска триггера 9

Для запуска триггера остаток ингредиента был изменен на 0. Перед этим был обновлен срок хранения, чтобы не нарушить ограничение таблицы `ingredient`.

```

restaurant_service=# SELECT
restaurant_service=#   d.id_dish,
restaurant_service=#   d.dish_name,
restaurant_service=#   d.is_available,
restaurant_service=#   i.id_ingredient,
restaurant_service=#   i.ingredient_name,
restaurant_service=#   i.stock_quantity,
restaurant_service=#   di.ingredient_amount
restaurant_service=# FROM dish d
restaurant_service=# JOIN dish_ingredient di
restaurant_service=#   ON di.id_dish = d.id_dish
restaurant_service=# JOIN ingredient i
restaurant_service=#   ON i.id_ingredient = di.id_ingredient
restaurant_service=# WHERE i.id_ingredient = 1
restaurant_service=# ORDER BY d.id_dish;
 id_dish | dish_name | is_available | id_ingredient | ingredient_name | stock_quantity | ingredient_amount
-----
(1 row)

 1 | Caesar Salad | f          | 1 | Tomato          | 0.000          | 0.150

restaurant_service=#
restaurant_service=# SELECT
restaurant_service=#   id_log,
restaurant_service=#   table_name,
restaurant_service=#   operation,
restaurant_service=#   row_id,
restaurant_service=#   description
restaurant_service=# FROM audit_log
restaurant_service=# ORDER BY id_log DESC
restaurant_service=# LIMIT 5;
 id_log | table_name | operation | row_id | description
-----
(1 row)

 3 | ingredient | UPDATE   | 1 | После изменения остатка ингредиента обновлена доступность блюд

restaurant_service=#
restaurant_service=# ROLLBACK;
ROLLBACK
restaurant_service=# █

```

Рис. 52. Проверка результата работы триггера 9

После изменения остатка блюдо стало недоступным: поле `is_available` получило значение `f`. Также появилась запись в таблице аудита.

Триггер 10. Проверка корректности назначения приготовления

Десятый триггер проверяет корректность назначения приготовления блюда.

```

restaurant_service=# CREATE OR REPLACE FUNCTION fn_cooking_assignment_validate()
restaurant_service=# RETURNS trigger
restaurant_service=# LANGUAGE plpgsql
restaurant_service=# AS $$
restaurant_service=# BEGIN
restaurant_service=#     IF NEW.ready_datetime < NEW.start_datetime THEN
restaurant_service=#         RAISE EXCEPTION 'Время готовности не может быть раньше времени начала приготовления';
restaurant_service=#     END IF;
restaurant_service=#     IF NOT EXISTS (
restaurant_service=#         SELECT 1
restaurant_service=#         FROM cook
restaurant_service=#         WHERE id_cook = NEW.id_cook
restaurant_service=#     ) THEN
restaurant_service=#         RAISE EXCEPTION 'Повар с ID % не найден', NEW.id_cook;
restaurant_service=#     END IF;
restaurant_service=#     IF NOT EXISTS (
restaurant_service=#         SELECT 1
restaurant_service=#         FROM head_chef
restaurant_service=#         WHERE id_head_chef = NEW.id_head_chef
restaurant_service=#     ) THEN
restaurant_service=#         RAISE EXCEPTION 'Шеф-повар с ID % не найден', NEW.id_head_chef;
restaurant_service=#     END IF;
restaurant_service=#     IF NEW.status IN ('assigned', 'in_progress')
restaurant_service=#     AND EXISTS (
restaurant_service=#         SELECT 1
restaurant_service=#         FROM cooking_assignment
restaurant_service=#         WHERE id_order_item = NEW.id_order_item
restaurant_service=#         AND status IN ('assigned', 'in_progress')
restaurant_service=#         AND id_assignment <> NEW.id_assignment
restaurant_service=#     )
restaurant_service=#     THEN
restaurant_service=#         RAISE EXCEPTION 'Для позиции заказа ID % уже есть активное назначение повару',
restaurant_service=#         NEW.id_order_item;
restaurant_service=#     END IF;
restaurant_service=#     RETURN NEW;
restaurant_service=# END;
restaurant_service=# $$;
CREATE FUNCTION
restaurant_service=#

```

Рис. 53. Код триггерной функции для триггера 10

Функция проверяет, что время готовности не раньше времени начала приготовления, что указанные повар и шеф-повар существуют, а также что для одной позиции заказа нет второго активного назначения.

```

restaurant_service=# DROP TRIGGER IF EXISTS trg_cooking_assignment_validate ON cooking_assignment;
NOTICE: trigger "trg_cooking_assignment_validate" for relation "cooking_assignment" does not exist, skipping
DROP TRIGGER
restaurant_service=#
restaurant_service=# CREATE TRIGGER trg_cooking_assignment_validate
restaurant_service=# BEFORE INSERT OR UPDATE ON cooking_assignment
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_cooking_assignment_validate();
CREATE TRIGGER
restaurant_service=#

```

Рис. 54. Создание триггера 10

Триггер создан перед операциями INSERT и UPDATE в таблице cooking_assignment.

```

restaurant_service=# CREATE TRIGGER trg_cooking_assignment_validate
restaurant_service=# BEFORE INSERT OR UPDATE ON cooking_assignment
restaurant_service=# FOR EACH ROW
restaurant_service=# EXECUTE FUNCTION fn_cooking_assignment_validate();
CREATE TRIGGER
restaurant_service=# SELECT
restaurant_service=#     oi.id_order_item,
restaurant_service=#     oi.id_order,
restaurant_service=#     oi.id_dish,
restaurant_service=#     c.id_cook,
restaurant_service=#     hc.id_head_chef
restaurant_service=# FROM order_item oi
restaurant_service=# CROSS JOIN cook c
restaurant_service=# CROSS JOIN head_chef hc
restaurant_service=# ORDER BY oi.id_order_item, c.id_cook, hc.id_head_chef
restaurant_service=# LIMIT 1;
 id_order_item | id_order | id_dish | id_cook | id_head_chef
-----+-----+-----+-----+-----
1 | 1 | 1 | 2 | 3
(1 row)
restaurant_service=#

```

Рис. 55. Проверка исходных данных для назначения приготовления

Перед проверкой были выбраны существующие идентификаторы позиции заказа, повара и шеф-повара.

```

restaurant_service=# INSERT INTO cooking_assignment (
restaurant_service(# id_assignment,
restaurant_service(# id_order_item,
restaurant_service(# id_cook,
restaurant_service(# id_head_chef,
restaurant_service(# status,
restaurant_service(# start_datetime,
restaurant_service(# ready_datetime
restaurant_service(# )
restaurant_service=# VALUES (
restaurant_service(# (SELECT COALESCE(MAX(id_assignment), 0) + 1 FROM cooking_assignment),
restaurant_service(# (SELECT MIN(id_order_item) FROM order_item),
restaurant_service(# (SELECT MIN(id_cook) FROM cook),
restaurant_service(# (SELECT MIN(id_head_chef) FROM head_chef),
restaurant_service(# 'assigned',
restaurant_service(# CURRENT_TIMESTAMP,
restaurant_service(# CURRENT_TIMESTAMP - INTERVAL '10 minutes'
restaurant_service=# );
ERROR: Время готовности не может быть раньше времени начала приготовления
CONTEXT: PL/pgSQL function fn_cooking_assignment_validate() line 4 at RAISE
restaurant_service=#

```

Рис. 56. Вызов действия, запускающего триггер 10

Попытка создать назначение, у которого время готовности меньше времени начала приготовления, была заблокирована.

4.4. Проверка списка созданных триггеров

Для итоговой проверки был выполнен запрос к представлению `information_schema.triggers`. Он показывает все триггеры, созданные в схеме `restaurant`, и события, на которые они срабатывают.

```

restaurant_service=# SELECT
restaurant_service=#     event_object_table AS table_name,
restaurant_service=#     trigger_name,
restaurant_service=#     event_manipulation AS event
restaurant_service=# FROM information_schema.triggers
restaurant_service=# WHERE trigger_schema = 'restaurant'
restaurant_service=# ORDER BY event_object_table, trigger_name, event_manipulation;

```

table_name	trigger_name	event
cooking_assignment	trg_cooking_assignment_validate	INSERT
cooking_assignment	trg_cooking_assignment_validate	UPDATE
customer_order	trg_customer_order_active_table	INSERT
customer_order	trg_customer_order_active_table	UPDATE
customer_order	trg_customer_order_status_transition	UPDATE
ingredient	trg_ingredient_update_dish_availability	UPDATE
order_item	trg_order_item_audit	DELETE
order_item	trg_order_item_audit	INSERT
order_item	trg_order_item_audit	UPDATE
order_item	trg_order_item_calculate_price	INSERT
order_item	trg_order_item_calculate_price	UPDATE
order_item	trg_order_item_check_stock	INSERT
order_item	trg_order_item_check_stock	UPDATE
order_item	trg_order_item_recalculate_total	DELETE
order_item	trg_order_item_recalculate_total	INSERT
order_item	trg_order_item_recalculate_total	UPDATE
purchase	trg_purchase_update_stock	INSERT
purchase	trg_purchase_validate	INSERT
purchase	trg_purchase_validate	UPDATE
(19 rows)		

```

restaurant_service=#

```

Рис. 57. Проверка списка созданных триггеров в схеме `restaurant`

В результате проверки было получено 19 строк. Это связано с тем, что часть триггеров создана сразу для нескольких событий, например `INSERT`, `UPDATE` и `DELETE`. Фактически в базе данных создано 10 триггеров, но в системном представлении каждое событие отображается отдельной строкой.

5. Выводы

В ходе выполнения лабораторной работы были разработаны и протестированы процедуры, функции и триггеры для базы данных `restaurant_service`. Были созданы три

процедуры, соответствующие индивидуальному варианту 13: вывод заказов заданного официанта за заданную дату, расчет стоимости заданного заказа и повышение оклада заданного сотрудника на 30% при изменении категории.

Также были созданы 10 триггеров, обеспечивающих контроль бизнес-логики базы данных. Триггеры запрещают создание второго активного заказа на один стол, контролируют переходы статусов заказа, автоматически рассчитывают стоимость позиций заказа и итоговую сумму заказа, проверяют наличие ингредиентов, фиксируют операции в таблице аудита, проверяют корректность закупок, обновляют остатки ингредиентов, изменяют доступность блюд и контролируют корректность времени приготовления.

Работа процедур и триггеров была проверена в консоли `psql`. В ходе проверки были выполнены как корректные операции, так и операции с ошибочными данными. Некорректные действия были заблокированы триггерами и процедурами, что подтверждает корректность реализованных ограничений и автоматизации.

Таким образом, цель лабораторной работы достигнута: получены практические навыки создания и использования процедур, функций и триггеров в PostgreSQL.